

操作系统

何柯

2026 年 1 月 7 日

目录

1	操作系统基础知识	5
1.1	基本概念	5
1.2	操作系统的历史演化	5
1.3	操作系统分类	6
1.4	操作系统结构	7
1.5	操作系统的启动和导引	9
1.6	应用程序的执行流程	10
2	操作系统运行环境与运行机制	11
2.1	硬件支持	11
2.2	中断与异常	12
2.3	系统调用	13
3	进程/线程模型	14
3.1	进程 (Process)	14
3.1.1	相关概念	14
3.1.2	状态变迁	16
3.1.3	进程控制	18
3.2	线程	19
3.2.1	相关概念	20
3.2.2	线程与进程的关系	20

目录	2
3.2.3 线程的实现方式	20
4 进程/线程调度	21
4.1 基本概念	21
4.2 调度算法设计准则	22
4.3 经典调度算法	22
5 进程同步机制	25
5.1 基本概念	25
5.2 实现方法	25
5.2.1 解决条件与问题综述	25
5.2.2 软件方法	25
5.2.3 硬件方法	27
5.2.4 信号量 (Semaphores)	29
5.2.5 常见锁类型的实现原理与伪代码	30
5.3 经典同步问题	33
5.3.1 生产者-消费者问题	33
5.3.2 读者-写者问题	34
5.3.3 哲学家就餐问题	39
5.3.4 睡眠理发师问题	40
5.4 进程通信 (IPC) 机制	42
6 死锁	44
6.1 基本概念	44
6.1.1 定义与产生原因	44
6.1.2 产生死锁的必要条件	44
6.1.3 资源分配图	44
6.2 处理策略	46
6.2.1 死锁预防: 破坏必要条件	46
6.2.2 死锁避免: 银行家算法 (安全序列)	46
6.2.3 死锁检测: 化简资源分配图	47
6.2.4 死锁解除	48

目录	3
7 内存管理	48
7.1 基础概念	48
7.2 内存分配方案	50
7.2.1 连续分配	50
7.2.2 离散分配	51
7.3 虚拟内存技术	52
7.4 优化技术	54
8 文件系统	54
8.1 文件系统基础	54
8.2 文件组织方式	55
8.2.1 空闲空间管理	55
8.2.2 接口设计	56
8.2.3 连续分配	57
8.2.4 链接分配	57
8.2.5 索引分配	59
8.3 文件目录与路径	59
8.3.1 文件链接	59
8.4 文件系统性能	59
8.5 虚拟文件系统 (VFS)	59
9 设备管理	60
9.1 基本概念	60
9.2 硬盘	62
9.2.1 机械硬盘	62
9.2.2 固态硬盘	62
9.3 I/O 系统层次架构	63
9.4 中断处理程序	64
9.4.1 上下文保护与恢复	64
9.4.2 上半部与下半部机制	65
9.5 设备独立内核软件技术	65
9.5.1 缓冲技术	65

目录	4
9.5.2 异步 I/O (AIO)	67
9.5.3 SPOOLing 技术 (假脱机)	68
9.5.4 设备分配与回收	68

1 操作系统基础知识

1.1 基本概念

操作系统的定义和作用：操作系统是管理硬件资源并为应用程序提供基础服务的核心系统软件。它让系统易于使用，且正确高效地运行，是用户与计算机硬件之间的接口；通过对硬件资源的管理与抽象，为上层软件提供统一、可用的运行环境。

操作系统的四大特征：

- 并发性 (Concurrency)：多个进程或线程可以同时运行。
- 共享性 (Sharing)：多个用户或进程可以共享系统资源。
- 虚拟性 (Virtualization)：提供抽象的资源视图，如虚拟内存、虚拟文件系统。
- 异步性 (Asynchrony)：事件和操作可以在不同时间发生，系统需要处理异步事件。

操作系统的四大功能：

- 进程管理
- 内存管理
- 文件系统管理
- 设备管理

1.2 操作系统的历史演化

1. 无操作系统：手动管理硬件资源，用户独占计算机资源，资源利用率低。
2. 单道批处理系统：引入作业调度和批处理。

批处理技术 是指计算机系统对一批作业自动进行处理的一种技术。用洗衣店举例：早期没有批处理时，你要手动伺候每一台洗衣机；而批处理可理解为给洗衣店安装了一个自动化系统：顾客把衣服送到店里，系

统会自动把衣服分类、安排洗衣机工作、烘干、折叠，最后通知顾客来取衣服。

3. 多道程序设计：让 CPU 在一个程序等待 I/O 时可以继续执行其他程序，从而显著提高资源利用率。
4. 分时系统：允许多个用户通过终端同时与计算机系统进行交互。操作系统通过将 CPU 时间分成细小的片段（称为时间片）并快速在多个用户任务间切换，给每个用户一种“独占”计算机资源的感觉，从而实现多用户共享。
5. 实时系统：在特定时间约束下执行任务，对外部事件作出快速响应；核心特征是时间敏感性，即系统必须在规定时间内完成某项任务或对某个事件作出响应。
6. 现代操作系统：支持多核处理器、移动设备与实时系统等。

1.3 操作系统分类

按应用场景分类

- 桌面操作系统：用户友好、图形界面、多媒体支持，如 Windows、macOS、Linux。
- 服务器操作系统：高性能、稳定性、安全性，如 Windows Server、Linux 发行版（如 Ubuntu Server、CentOS）。
- 移动操作系统：轻量级、触摸优化，如 Android、iOS。
- 嵌入式操作系统：资源受限、实时性强，如 FreeRTOS、VxWorks。

按架构特征分类

- 单用户系统：简化管理，提高性能，如 DOS。
- 多用户系统：资源共享与安全性，如 UNIX、Linux。
- 单任务系统：简单高效，如早期 DOS。
- 多任务系统：提高资源利用率，如 Windows、Linux。

按内核结构分类 什么是操作系统的内核：负责管理所有资源，并为上层应用提供服务。没有内核，软件就无法指挥硬件干活。

- 宏内核结构：所有操作系统服务都运行在内核空间，如传统 UNIX、Linux。
- 微内核结构：只保留最基本功能在内核空间，如 MINIX、QNX。
- 混合内核结构：结合宏内核与微内核优点，关键服务在内核，其他在用户态，如 Windows NT、macOS。

1.4 操作系统结构

操作系统是复杂的大型软件，为了控制设计复杂度，通常采用“策略与机制分离”的原则，并利用模块化、抽象、分层等方法进行管理。

常见内核结构

1. 简单结构 (Simple Structure)

- 代表系统：MS-DOS。
- 特点：结构简单，没有清晰的模块划分。

2. 宏内核结构 (Monolithic Kernel)

- 代表系统：初始 UNIX、Linux、传统 Unix。
- 定义：系统调用之下、物理硬件之上的部分均为内核，所有服务在内核空间运行；内核通过系统调用提供文件系统、CPU 调度、内存管理等功能。
- 优点：运行效率高、通信效率高。
- 缺点：结构难以理解与维护；系统复杂性增加时可靠性和安全性风险更高。

3. 模块化结构 (Modular Structure)

- 代表系统：现代操作系统（如 Unix、Linux、Windows）。例如 Solaris 由核心内核及 7 个可加载内核模块构成。
- 定义：模块功能独立且封装，具有良好定义的接口；Linux 也常被描述为“宏内核 + 模块化设计”。
- 优点：易于理解，效率高。
- 缺点：全局函数使用多可能造成访问控制困难；结构不清晰会导致可理解性、可维护性与可移植性下降；存在潜在性能退化。

4. 微内核结构 (Microkernel)

- 代表系统：Mach、Minix、QNX、L4。
- 定义：基于客户/服务器模型，由微内核与核外用户空间的服务器进程构成。
- 内核功能：仅保留最核心功能，如进程管理、内存管理与通信。
- 通信机制：通过消息传递 (Message Passing) 实现客户程序与服务程序通信。
- 优点：

易于扩展。

易于从一个硬件架构移植到另一个。

更可靠（内核代码更少）。

- 缺点：用户空间到内核空间的通信增加系统开销，早期性能不及宏内核。

1.5 操作系统的启动和导引

操作系统启动是一个分阶段的过程，旨在解决“如何让一台空白的计算机运行起复杂的操作系统”这一核心问题。该过程主要分为三个阶段：

1. 阶段 1：硬件自举 (**Hardware Bootstrapping**)——解决 CPU 执行起点问题。

核心目标 让上电后“空白”的 CPU 具备基本执行能力，完成硬件自检，并找到下一阶段程序。

ROM 启动 CPU 上电后固定跳转到主板 ROM（只读存储器）的特定地址（如 RISC-V 的 `0x1000`）执行固化的初始程序。

BIOS 任务

- (a) 检查硬件：检测内存、硬盘等硬件是否正常。
 - (b) 初始化硬件：启动内存控制器、显示器和 I/O 设备。
 - (c) 加载 Bootloader：从存储设备（如硬盘引导扇区）读取 Bootloader 到内存，并移交 CPU 控制权。
2. 阶段 2：引导加载器 (**Bootloader**)——搭建硬件与内核的桥梁。

核心目标 作为中间层，解决 BIOS 功能简单无法直接加载复杂内核的问题，为内核运行准备环境。

环境准备 隐藏硬件差异，设置内存布局，初始化栈空间。

加载内核 Bootloader 识别文件系统（如 ext4、FAT32），将操作系统内核文件（如 `vmlinuz`）从硬盘读入内存指定区域。

移交控制 将硬件配置信息传递给内核，并将 CPU 控制权移交给操作系统内核。

3. 阶段 3：内核初始化 (**Kernel Initialization**)——建立完整的操作系统环境。

核心目标 从单程序执行过渡到多任务系统，建立资源管理体系。

搭建管理体系

- (a) 内存管理：建立页表，启用 MMU，实现虚实地址映射。
- (b) 进程管理：初始化进程调度队列（就绪/阻塞队列），支持并发。
- (c) 文件系统：挂载根文件系统。

加载驱动与中断 加载网卡、显卡等驱动，建立中断向量表。

启动用户环境 启动第一个用户态进程（如 Linux 的 `init/systemd`），加载登录界面并等待用户登录。

1.6 应用程序的执行流程

1. 加载：启动程序时，由操作系统加载器将可执行文件加载到内存。
2. 运行：操作系统创建新进程，并将 CPU 控制权交给该进程，开始执行程序代码。
3. 系统调用：程序需要操作系统服务（如文件操作、内存分配）时，通过系统调用接口向内核发出请求。
4. 中断处理：发生中断（如 I/O 完成、定时器中断）时，CPU 暂停当前进程并执行中断处理程序。
5. 退出：程序执行完成或需停止时，执行退出：释放资源、关闭文件并通知父进程等。

2 操作系统运行环境与运行机制

2.1 硬件支持

CPU 重要寄存器的作用：

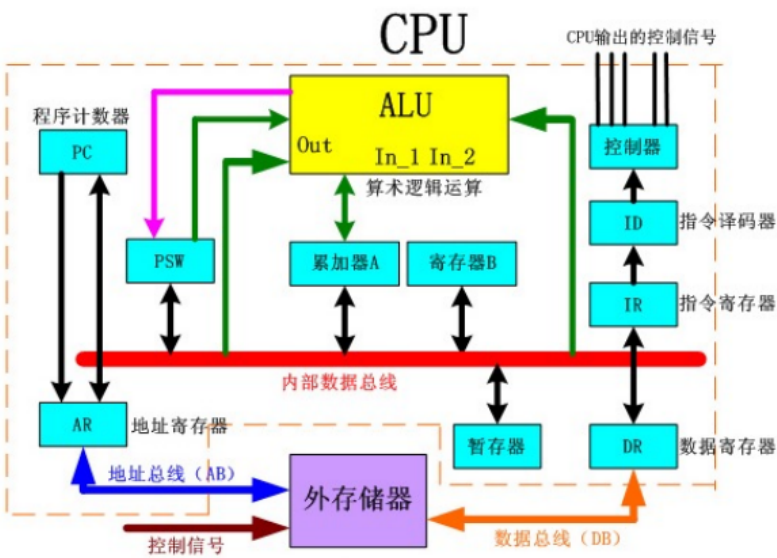


图 1: CPU 重要寄存器示意图

- 程序计数器 (**Program Counter, PC**): 存储下一条将要执行的指令地址。
- 指令寄存器 (**Instruction Register, IR**): 记录最近取出并解码的指令。
- 程序状态字 (**PSW**): 记录处理器运行状态，如条件码、模式与控制位；其中某些位会影响处理器行为（如中断允许位、处理器状态位）。
- 通用寄存器 (**General Purpose Registers**): 存储临时数据与计算结果。
- 控制寄存器 (**Control Registers**): 只能由操作系统使用，用于控制与配置处理器特性（如模式切换、内存管理相关配置）。

运行模式：

表 1: 内核态与用户态对比

对比维度	内核态 (Kernel Mode)	用户态 (User Mode)
程序类型	操作系统程序执行	用户程序执行
指令权限	使用全部指令	禁止使用特权指令
资源访问	使用全部系统资源（包括整个存储区域）	只允许用户程序访问自己的存储区域

CPU 特权级切换触发条件：

- 应用程序调用操作系统提供的系统调用。
- 应用程序执行一条指令触发异常。
- 应用程序执行过程中收到来自外设的中断。

2.2 中断与异常

因为中断处理程序运行在核心态（特权模式），而被打断的程序通常运行在用户态（非特权模式），所以需要硬件支持完成“用户态 → 核心态”的切换。

概念：

- 中断：来自 CPU 外部的的事件，如 I/O 设备请求服务。
- 异常：来自 CPU 内部的事件，如除零错误、非法指令等。

中断向量表：它是存储在内存固定区域的数据结构，表中每一项存放中断处理程序入口地址（即程序计数器 PC 的值）与状态字（PSW）。

中断处理过程：

1. 保存现场：保存当前进程状态，包括 PC、PSW 与其他寄存器内容，以便中断处理完成后恢复执行。
2. 中断服务程序执行：根据中断向量表定位对应中断处理程序，并执行相应处理逻辑。
3. 恢复现场：恢复 PC、PSW 与其他寄存器内容，继续被打断的执行流。

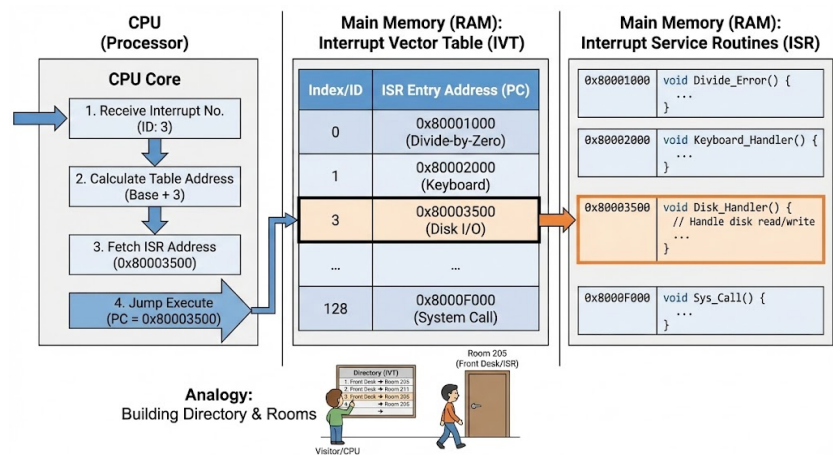


图 2: 中断向量表示意图

2.3 系统调用

系统调用是操作系统提供服务的编程接口。通常用高级语言（如 C/C++）编写；大多数程序通过高级应用程序编程接口（API）而不是直接使用系统调用。最常见的三个 API：Windows 的 Win32 API，基于 POSIX 的系统（包括几乎所有版本的 UNIX、Linux 与 macOS）的 POSIX API，以及 Java 虚拟机（JVM）的 Java API。

作用：用户态请求内核服务，是用户程序与操作系统之间的桥梁。

实现机制：

1. 访管指令 (Trap Instruction)：用户程序执行特殊指令（如 x86 的 `int` 指令），触发软中断并切换到内核态。
2. 系统调用表 (System Call Table)：内核维护系统调用表，存储各系统调用处理程序地址。
3. 系统调用号 (System Call Number)：用户程序通过寄存器或栈传递系统调用号，内核据此查表定位处理程序。

3 进程/线程模型

3.1 进程 (Process)

3.1.1 相关概念

定义：正在执行的程序，具有动态性与生命周期（从创建到消亡）。执行时需要处理机资源（资源分配的基本单位，是程序的一次执行）。

进程控制块 (PCB)：管理程序运行状态的数据结构。

PCB 主要内容：

- 进程状态：运行、等待、就绪等。
- 进程标识符：唯一标识进程的 ID。
- 进程队列指针：记录 PCB 队列中下一个 PCB 的地址。
- 程序与数据地址：进程程序与数据在内存或外存中的存放地址。
- 进程优先级：反映进程获得 CPU 的优先级别。
- CPU 现场保护区：CPU 现场信息存放区域，包括通用寄存器、PC、PSW 等。
- 通信信息：进程与其他进程交换信息时记录的有关信息。
- 家族关系：父子进程标识等。
- 资源信息：进程拥有的资源及其使用情况。

程序转换为进程的过程：

1. 程序加载：操作系统将可执行文件加载到内存并分配必要资源。
2. 创建 PCB：为新进程创建进程控制块并初始化信息。
3. 设置初始状态：将进程状态置为就绪，等待调度。
4. 调度执行：调度进入运行状态并开始执行。

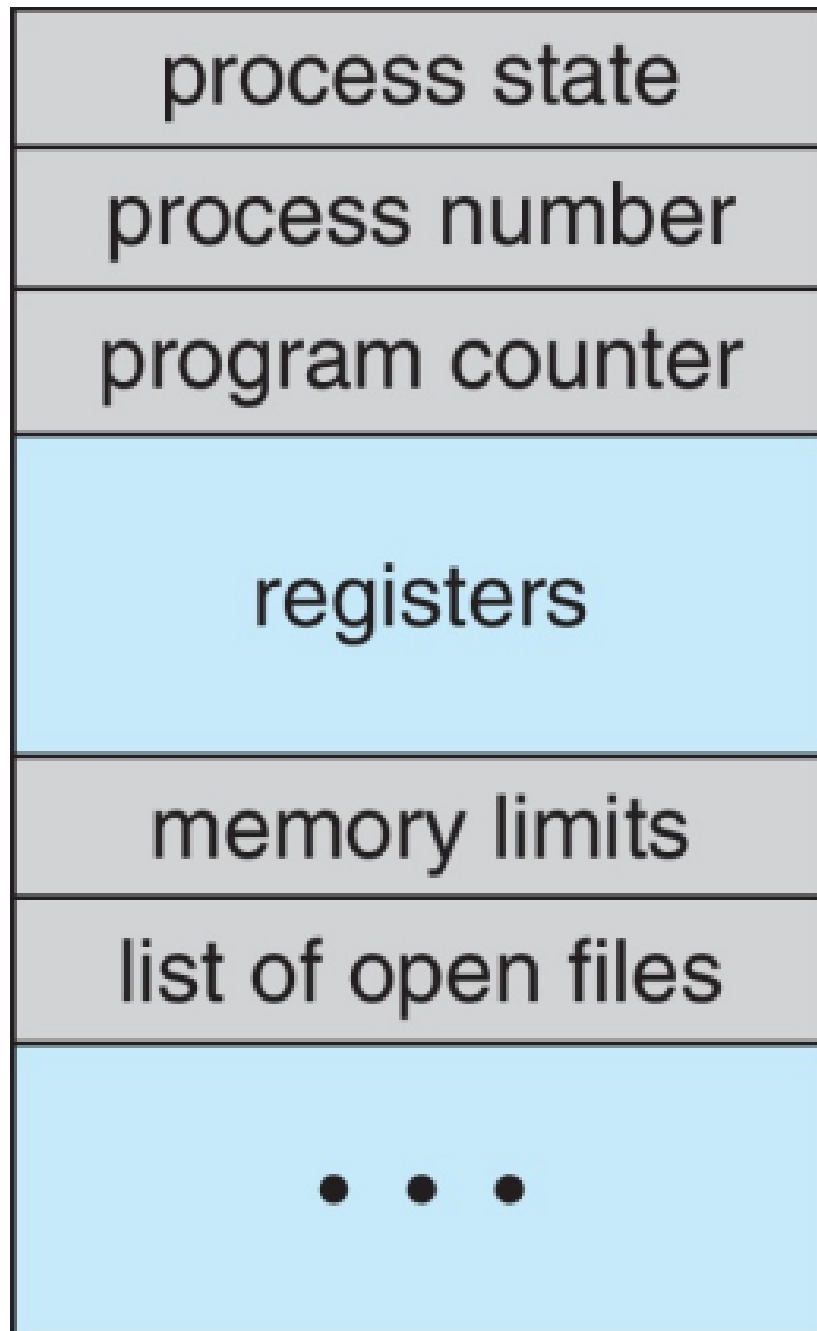


图 3: 进程控制块 PCB 示意图

3.1.2 状态变迁

进程的生命周期包含一系列状态转换。最基本模型包含五个状态：新建 (New)、就绪 (Ready)、运行 (Running)、等待 (Waiting)、终止 (Terminated)。基本主干线路为：新建 → 就绪 → 运行 → 终止。

- **新建 New**: 进程被创建
- **运行 Running**: 指令执行时的状态
- **等待 Waiting**: 进程等待某些事件发生时的状态
- **就绪 Ready**: 进程等待获得 CPU
- **终止 Terminated**: 进程正常或异常结束时的状态



图 4: 进程五状态变迁示意图

1. 基本状态转换:

- 就绪 → 运行 (**Scheduler Dispatch**): 调度程序选择该进程执行, 进程获得 CPU 资源。
- 运行 → 就绪 (**Interrupt**):

时间片用完: 进程分配的 CPU 时间片耗尽, 必须让出 CPU。

被抢占: 更高优先级进程进入就绪队列。

- 运行 → 等待 (**I/O or Event Wait**): 这是主动过程。进程请求 I/O 或等待事件, 暂时无法继续执行。

- 等待 → 就绪 (I/O or Event Completion): 这是被动过程。I/O 完成或事件发生后进程被唤醒, 进入就绪队列。
- 运行 → 终止 (Exit): 进程执行完毕或遇到错误退出。

2. 挂起状态 (Suspend) 的引入: 为解决内存紧张或用户干预等问题, 引入“挂起”状态。挂起状态下, 进程保存在磁盘 (外存) 上, 不占用内存资源, 也不占用 CPU 时间。

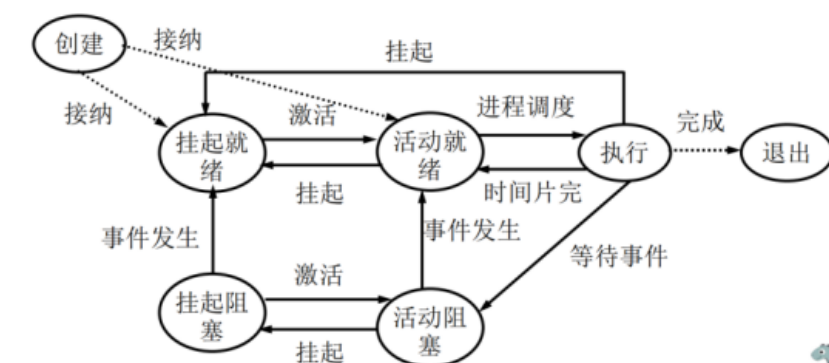


图 5: 引入挂起状态后的进程状态扩展图

3. 扩展后的状态转换（七状态模型）: 引入挂起后, 增加了挂起就绪 (Suspend Ready) 与挂起阻塞/等待 (Suspend Waiting) 两个新状态:

- 挂起 (Suspend):

活动就绪 → 挂起就绪: 内存不足或系统负荷过高时, 将就绪进程换出到磁盘。

活动阻塞 → 挂起阻塞: 进程等待事件且内存不足时, 将其换出。

- 激活 (Activate):

挂起就绪 → 活动就绪: 内存空闲或该进程优先级较高时将其调入内存。

挂起阻塞 → 活动阻塞： 调入内存后仍需等待事件。

- 挂起状态下事件发生：

挂起阻塞 → 挂起就绪： 外存等待的事件发生后，虽仍在磁盘上，但状态变为“就绪”，被激活即可运行。

3.1.3 进程控制

进程控制是操作系统内核通过原语 (**Primitive**) 与系统调用 (**System Call**) 对进程生命周期进行管理的核心机制，主要包括：

1. 进程创建 (**Creation**) —— **fork()** 与 **exec()**

- 机制：创建具有唯一标识符 (PID) 的新进程，并分配 PCB。
- **fork()**:

用于创建子进程。

子进程是父进程的副本（复制地址空间），但拥有独立 **PID**。

- **exec()**:

通常在 **fork()** 之后使用。

2. 进程终止 (**Termination**) —— **exit()**

- 触发条件：任务完成正常退出，或因错误（如非法内存访问）被强制退出。
- 处理流程：

进程执行 **exit()** 请求删除自身。

操作系统回收该进程占用的资源（内存、文件等）。

3. 进程阻塞 (Blocking) ——wait()

- 定义：进程等待事件（如 I/O 完成、资源可用）而暂停执行，主动放弃 CPU。
- wait()：

父进程调用以等待子进程终止。

若子进程未结束，父进程进入阻塞状态，直到接收子进程退出信号。

- 内部动作：保存 CPU 现场到 PCB，将状态置为等待/阻塞，并插入等待队列，调度其他进程。

4. 进程唤醒 (Waking up)

- 定义：等待事件发生（如 I/O 结束、子进程退出）时，由发现者将其唤醒。
- 处理流程：

在等待队列中找到该进程 PCB。

将其移出等待队列。

将进程状态改为就绪 (Ready)。

3.2 线程

3.2.1 相关概念

定义：进程内的一个执行单元，是 CPU 调度和分派的基本单位。线程是比进程更小的独立运行单位，一个进程可以包含多个线程。

为什么要引入线程：

- 单线程应用的瓶颈问题。
- 应用程序中的多个任务可通过多个线程实现。
- 线程创建是轻量级的，而进程创建是重量级的。
- 多线程可简化代码、提高效率。
- 内核通常也是多线程的。

多线程的优势：

- 响应性：部分阻塞时仍可继续执行，对用户界面尤其重要。
- 资源共享：同一进程内线程共享内存空间与文件描述符等资源。
- 经济性：创建与销毁线程的开销远小于进程。
- 可扩展性：可利用多核架构优势。

3.2.2 线程与进程的关系

- 线程属于进程，可视为进程内部的执行路径。
- 线程共享进程资源（内存空间、文件描述符等）。
- 线程间通信通常更简单，线程切换开销也更小。

3.2.3 线程的实现方式

1. 用户级线程 (User-Level Threads, ULT)

- 线程管理完全在用户空间进行，内核不感知线程存在。
- 优点：切换开销小，创建与销毁快。
- 缺点：无法利用多核；阻塞一个线程可能阻塞整个进程。

2. 内核级线程 (Kernel-Level Threads, KLT)

- 线程由内核管理，内核知道每个线程。
- 优点：可利用多核；一个线程阻塞不影响其他线程。
- 缺点：切换开销较大，创建与销毁较慢。

3. 混合线程模型 (Hybrid Thread Model)

- 结合用户级与内核级线程的优点。
- 用户级线程映射到内核级线程上，允许多个用户级线程共享一个内核级线程。
- 优点：既能利用多核，又能减少线程切换开销。

简单理解

- 用户级线程 → 门经理（用户态线程库）自己管，不告诉总部（内核）。
- 内核级线程 → 总部（内核）直接管理每个小组（线程）。
- 混合线程模型 → 门经理与总部共同管理。

4 进程/线程调度

4.1 基本概念

调度：协调各请求对资源的使用的方法。

多级调度：分为长程调度（作业调度）、中程调度（交换调度）和短程调度（CPU 调度）。

- 长程调度 (Long-Term Scheduling): 决定哪个程序被接纳并创建进程放入就绪队列; 控制多道程序数量。
- 中程调度 (Medium-Term Scheduling): 负责将部分进程从内存换出到外存 (挂起) 以释放内存; 资源充足时再换入 (激活)。
- 短程调度 (Short-Term Scheduling): 决定哪个进程获得 CPU 使用权并执行指令, 实现并发。

抢占式调度与非抢占式调度:

- 抢占式调度 (Preemptive Scheduling): 操作系统可中断正在运行进程, 将 CPU 分配给另一进程; 允许高优先级进程优先执行, 提高响应速度。
- 非抢占式调度 (Non-Preemptive Scheduling): 进程获得 CPU 后一直运行到自愿放弃 CPU (进入等待或终止); 实现简单, 但可能导致低优先级进程长期得不到执行。

上下文切换: 操作系统从一个进程切换到另一个进程时, 需要保存当前进程上下文并加载新进程上下文, 包括寄存器、PC、内存映射等信息。

4.2 调度算法设计准则

- CPU 利用率 (CPU Utilization): CPU 被有效利用的程度。
- 吞吐量 (Throughput): 单位时间内完成的进程数量。
- 周转时间: 任务提交到完成所经历的时间, 包括等待与执行时间。 周转时间 = 等待时间 + 运行时间; 带权周转时间 = 周转时间 / 运行时间。
- 等待时间: 任务在就绪队列中等待的总时间。
- 响应时间: 任务提交到首次开始执行所经历的时间。

4.3 经典调度算法

- 先来先服务 (FCFS): 先来先执行。
- 短作业优先 (SJF) (非抢占): 选择 CPU 执行时间最短的进程。

- 最短剩余时间优先 (SRTN) (SJF 抢占版)：考虑到达时间，始终选择剩余执行时间最短的进程。

■ 考虑进程的到达时间，且允许抢占

Process	ArrivalTime	Burst Time
P ₁	0	8
P ₂	1	4
P ₃	2	9
P ₄	3	5

■ 抢占式 SJF 甘特图

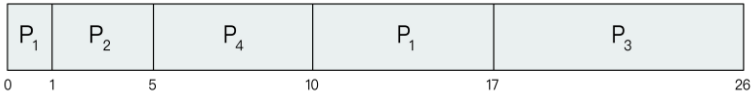


图 6: 最短剩余时间优先 (SRTN) 调度示意图

- 最高响应比优先 (HRRN) (非抢占)：响应比 = (等待时间 + 服务时间)/服务时间，选择响应比最高的进程。
- 时间片轮转 (RR)：为每个进程分配固定时间片，用完后放回就绪队列末尾。
- 优先级调度：按优先级选择进程执行，可抢占或非抢占。

多级队列与多级反馈队列 多级队列：按特征将进程划分到不同队列，每队列使用自己的调度算法。多级反馈队列：结合时间片轮转与优先级调度，允许进程在队列间动态迁移。

多处理器调度亲和性 多处理器调度的亲和性是指进程倾向在特定处理器或处理器集合上运行的特性。原因：进程/线程之前在这些处理器上运行过，相关数据可能仍在缓存中；若再次在同一处理器上运行可利用缓存数据，提高性能。

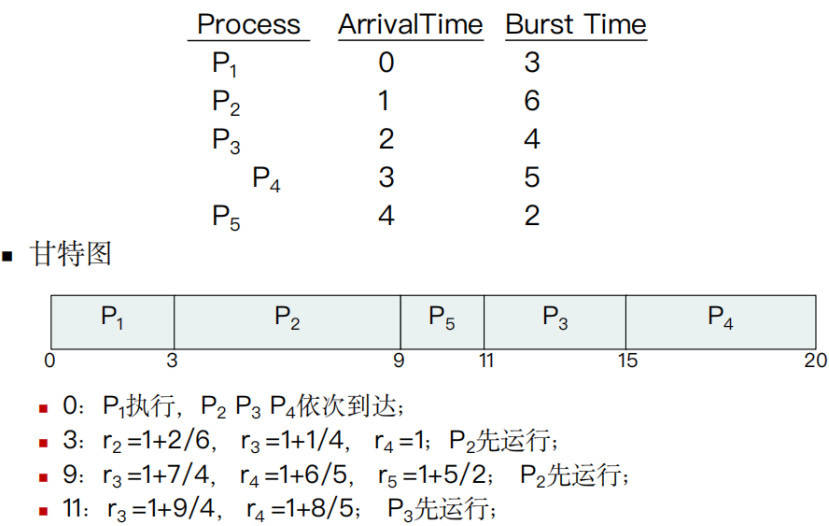


图 7: 最高响应比优先 (HRRN) 调度示意图

表 2: 多级队列与多级反馈队列调度算法对比

对比维度	多级队列调度算法	多级反馈队列调度算法
进程分类方式	静态分类：进程一旦入队，终身属于该队列	动态分类：进程会根据执行情况在队列间迁移
队列间迁移	无迁移：进程入队后不会改变所属队列	有迁移：时间片耗尽可能降级；长期未调度可能升级（防止饥饿）
灵活性	低：队列划分固定，难以适应动态变化	高：动态调整队列，更贴合实际运行场景
饥饿问题	易出现：低优先级队列可能长期得不到调度	不易出现：通过迁移机制避免饥饿
实现复杂度	较低：按类型划分队列并配置算法	较高：需设计迁移规则与优先级调整逻辑
典型应用场景	进程类型明确且稳定的系统	通用操作系统（如 Linux、Windows）

5 进程同步机制

5.1 基本概念

竞态条件：多个进程/线程同时进入临界区，执行结果与访问发生的特定顺序有关。

临界区：访问共享资源的一段代码。

原子操作：原子操作是实现操作系统原语 (Primitive) 的基础；一个操作（或一系列操作）在执行过程中不会被任何机制（中断、信号、其他线程）打断。

5.2 实现方法

5.2.1 解决条件与问题综述

解决临界区问题需要满足的条件：

- 互斥：同一时刻只能有一个进程/线程进入临界区。
- 进展性：若无人处于临界区且有人想进入，必须能选出一个进入。
- 有界等待：进程提出进入请求后，其他进程进入临界区的次数必须有限。

两种常见并发问题：

- 共享资源引起的冲突。
- 占有 CPU 导致的饥饿问题。

5.2.2 软件方法

软件方法是指仅通过代码逻辑（变量标记、循环判断）实现互斥，而不依赖特殊硬件指令。

1. 单标志法 (Strict Alternation)

- 机制：设置一个公用整型变量 `turn`，指示允许进入临界区的进程编号。
- 核心逻辑：

```
1 // 进程 P0 // 进程 P1
2 while (turn != 0); while (turn != 1);
3 // 临界区 // 临界区
4 turn = 1; turn = 0;
```

- 缺点：违背“进展性”。若进程 0 不想进入临界区，它不会把 `turn` 改为 1，导致进程 1 即使想进也无法进入。

2. 双标志法先检查 (Check then Set)

- 机制：设置布尔数组 `flag[]`。进程先检查对方是否想进，若不想，则设置自己标志并进入。
- 核心逻辑：

```
1 while (flag[j]); // 1. 先检查对方是否想进
2 flag[i] = true; // 2. 后设置自己想进
3 // 临界区
4 flag[i] = false; // 3. 退出时修改标志
```

- 缺点：违背“互斥”。若两个进程同时检测到 `flag` 为 `false`，会同时设置标志并进入临界区。

3. 双标志法后检查 (Set then Check)

- 机制：先设置 `flag[i]=true`，再检测对方。
- 核心逻辑：

```
1 flag[i] = true; // 1. 先设置自己想进
2 while (flag[j]); // 2. 后检查对方是否想进
3 // 临界区
4 flag[i] = false; // 3. 退出
```

- 缺点：可能导致死锁。若两个进程同时举旗，然后互相等待对方撤旗，会无限等待。

4. 皮特森算法 (Peterson's Algorithm)

- 机制：结合双标志法与单标志法：flag[] 表示意愿，turn 表示谦让。
- 核心逻辑：

```
1 flag[i] = true; // 1. 我想进（意愿）
2 turn = j; // 2. 让你先进（谦让）
3 // 3. 检查：如果你想进 且 确实轮到你，那我就等
4 while (flag[j] && turn == j);
5 // 临界区
6 ...
7 flag[i] = false; // 4. 退出
```

- 评价：解决互斥、进展性与有界等待问题，是经典软件方案。

5.2.3 硬件方法

硬件方法利用 CPU 提供的特殊指令从底层保证某些操作的原子性。

1. 关中断 (Disable Interrupts)

- 原理：进入临界区前关闭中断，避免进程切换，从而独占 CPU。
- 伪代码：

```
1 disable_interrupts(); // 关中断
2 // ... 临界区代码 ...
3 enable_interrupts(); // 开中断
```

- 缺点：不适用于多处理器；将关中断权限交给用户进程极其危险；限制 CPU 处理其他紧急任务能力。

2. Test-and-Set (TSL) 指令

- 机制：硬件保证读取旧值与设置新值原子执行。
- 核心逻辑：

```
1 // 硬件原子指令定义
2 bool TestAndSet(bool *target) {
3     bool rv = *target; // 1. 读旧值
```

```
4      *target = true; // 2. 设新值 (上锁)
5      return rv; // 3. 返回旧值
6  }
7
8  // 进程逻辑
9  while (TestAndSet(&lock)); // 自旋等待: 只要返回 true 就一直循环
10 // ... 临界区 ...
11 lock = false; // 解锁
```

- 特点: 适用于多处理器, 但会导致忙等待 (**Busy Waiting**), 浪费 CPU 时间。

3. Swap (Exchange) 指令

- 机制: 原子交换两个内存位置的值。
- 核心逻辑:

```
1 // 硬件原子指令定义
2 void Swap(bool *a, bool *b) {
3     bool temp = *a;
4     *a = *b;
5     *b = temp;
6 }
7
8 // 进程逻辑
9 key = true;
10 while (key == true) {
11     Swap(&lock, &key); // 尝试交换
12 }
13 // ... 临界区 ...
14 lock = false; // 解锁
```

- 特点: 简单易实现, 适用于多处理器, 但同样存在忙等待问题, 可能导致饥饿。

5.2.4 信号量 (Semaphores)

信号量 (Semaphore) S 是包含一个整数值对象，只能通过两个原子操作 `wait()` 与 `signal()` 访问，也称为 $P()$ 与 $V()$ 。可以把信号量理解为只通过原子操作控制的整型变量。

```
1 // P操作 (Wait) // V操作 (Signal)
2 wait(S) { signal(S) {
3     while (S <= 0); // 自旋等待(或阻塞) S = S + 1; // 释放资源
4     S = S - 1; // 占用资源 }
5 }
```

信号量机制主要用于两类场景：实现互斥与实现同步。

1. 利用信号量实现互斥 (Mutual Exclusion)

- 原理：将信号量 S 初始化为 1（代表一把锁）。
- 流程：

进入临界区前执行 $P(S)$ ：若 $S = 1$ 则变为 0 并进入；若 $S = 0$ 则等待。

- 伪代码结构：

```
1 semaphore mutex = 1; // 初始化互斥信号量为 1
2
3 P(mutex); // 1. 关门：申请资源
4 // -----
5 // 临界区 (Critical Section)
6 // (读写共享数据)
7 // -----
8 V(mutex); // 2. 开门：释放资源
```

2. 利用信号量实现同步 (Synchronization)

- 原理：将信号量 S 初始化为 0（资源初始不可用或前序尚未完成）。

- 目标：保证进程 P_1 的语句 x 执行完后，进程 P_2 才能执行语句 y （前趋关系 $x \rightarrow y$ ）。
- 流程：

前操作进程 P_1 ：完成后执行 $V(S)$ ，令 S 变为 1 通知对方。

- 伪代码结构：

```
1 semaphore S = 0; // 初始化同步信号量为 0
2
3 // 进程 P1（先执行） // 进程 P2（后执行）
4 { {
5     代码 x; P(S); // 检查P1是否完成?
6     V(S); // 通知P2 代码 y;
7 } }
```

同步：强制让两个原本各跑各的进程按照规定先后顺序执行，以保证后续进程拿到前序进程准备好的数据。

总结：

- 互斥： S 初值为 1，P/V 操作在同一个进程中成对出现（夹住临界区）。
- 同步： S 初值为 0，P/V 操作在两个不同进程中成对出现（一个发信号，一个等信号）。

5.2.5 常见锁类型的实现原理与伪代码

根据线程等待锁的方式（忙等待 vs 睡眠等待）及控制粒度不同，锁的内部实现逻辑存在差异。

1. 自旋锁 (Spinlock)

- 机制：获取锁失败时线程不挂起，而是循环检查锁是否空闲（忙等待）。

- 底层原理:通常基于硬件的 Test-and-Set (TSL) 或 Compare-and-Swap (CAS)。
- 优点: 无上下文切换开销, 获取锁速度快。
- 缺点: 持锁线程不释放时, 等待线程空转浪费 CPU。
- 适用场景: 多核且临界区极短 (短于上下文切换) 的情况。
- 伪代码实现:

```
1 // lock_flag: 0为空闲, 1为占用
2 void spin_lock(int *lock_flag) {
3     while (TestAndSet(lock_flag) == 1) {
4         ; // 空转
5     }
6 }
7
8 void spin_unlock(int *lock_flag) {
9     *lock_flag = 0;
10 }
```

2. 互斥锁 (Mutex)

- 机制: 获取锁失败时线程主动放弃 CPU, 进入阻塞 (Sleep) 并加入等待队列; 锁释放后由系统唤醒。
- 优点: 不忙等待, 节省 CPU, 适合长时间持锁。
- 缺点: 涉及睡眠/唤醒与上下文切换, 开销较大。
- 适用场景: 临界区较长或持锁时间难预测。
- 伪代码实现:

```
1 void mutex_lock(Mutex *m) {
2     if (TestAndSet(&m->flag) == 1) {
3         add_to_queue(m->wait_queue, current_thread);
4         block();
5     }
6 }
```

```
7
8 void mutex_unlock(Mutex *m) {
9     if (is_empty(m->wait_queue)) {
10         m->flag = 0;
11     } else {
12         Thread t = remove_from_queue(m->wait_queue);
13         wakeup(t);
14     }
15 }
```

3. 读写锁 (RWLock)

- 机制：区分读者与写者：读-读不互斥、读-写互斥、写-写互斥。
- 伪代码实现：

```
1 Semaphore rw_mutex = 1; // 写锁
2 Semaphore count_mutex = 1; // 计数器锁
3 int readers = 0;
4
5 // --- 读者 ---
6 void read_lock() {
7     P(count_mutex);
8     readers++;
9     if (readers == 1) {
10         P(rw_mutex);
11     }
12     V(count_mutex);
13 }
14
15 void read_unlock() {
16     P(count_mutex);
17     readers--;
18     if (readers == 0) {
19         V(rw_mutex);
```



```
20     }  
21     V(count_mutex);  
22 }  
23  
24 // --- 写者 ---  
25 void write_lock() {  
26     P(rw_mutex);  
27 }
```

条件变量：一种用于“等待”的同步机制，通常与互斥锁结合使用；允许线程在条件不满足时等待，并在条件满足时被唤醒继续执行。

5.3 经典同步问题

5.3.1 生产者-消费者问题

问题简述：也称有界缓冲区问题：生产者将数据放入有限缓冲区，消费者取出数据消费；需保证缓冲区满时生产者不继续生产，缓冲区空时消费者不继续消费。

解决要点：既要保证同步（满/空约束），也要保证互斥（保护缓冲区）。

信号量定义：

- semaphore mutex = 1; 互斥信号量，保护缓冲区
- semaphore empty = n; 空闲缓冲区数量
- semaphore full = 0; 已占用缓冲区数量

生产者进程：

```
1 producer() {  
2     while(true) {  
3         produce_item();  
4         P(empty);
```

```
5      P(mutex);  
6      put_item();  
7      V(mutex);  
8      V(full);  
9  }  
10 }
```

消费者进程:

```
1  consumer() {  
2      while(true) {  
3          P(full);  
4          P(mutex);  
5          get_item();  
6          V(mutex);  
7          V(empty);  
8          consume_item();  
9      }  
10 }
```

关键点:

- $P(\text{empty})$ 与 $P(\text{full})$ 必须在 $P(\text{mutex})$ 之前, 否则可能死锁。
- $V(\text{mutex})$ 应尽快执行, 再执行 $V(\text{empty})/V(\text{full})$ 。

5.3.2 读者-写者问题

问题简述: 多个读者与写者访问共享数据。读者可并发读, 写者需独占。

解决要点: 三种常见策略: 读者优先、写者优先、公平策略。

读者优先 信号量定义:

- semaphore rw_mutex = 1;

- semaphore count_mutex = 1;
- int readers = 0;

读者逻辑:

```
1 void read_lock() {
2     P(count_mutex);
3     readers++;
4     if (readers == 1) {
5         P(rw_mutex);
6     }
7     V(count_mutex);
8 }
9
10 void read_unlock() {
11     P(count_mutex);
12     readers--;
13     if (readers == 0) {
14         V(rw_mutex);
15     }
16     V(count_mutex);
17 }
```

写者逻辑:

```
1 void write_lock() {
2     P(rw_mutex);
3 }
4
5 void write_unlock() {
6     V(rw_mutex);
7 }
```

存在问题: 读者优先可能导致写者饥饿。

写者优先 核心思想: 一旦有写者等待, 就不允许新的读者进入。

信号量定义：

- semaphore reader_count_lock = 1;
- semaphore writer_count_lock = 1;
- semaphore read_try = 1;
- semaphore write_lock = 1;
- int reader_count = 0;
- int writer_count = 0;

表 3: 写者优先方案的信号量组件说明

组件名称	作用
reader_count	记录当前读者数量（需互斥保护）
writer_count	记录等待/执行写者数量（需互斥保护）
reader_count_lock	保护 reader_count 的互斥锁
writer_count_lock	保护 writer_count 的互斥锁
read_try	写者优先的“控制门”，写者可用其阻塞新读者
write_lock	核心互斥锁，实现读-写互斥与写-写互斥

读者逻辑：

```
1 void reader() {
2     P(read_try); // (1) 检查是否有写者在等
3     P(reader_count_lock);
4     reader_count++;
5     if (reader_count == 1) {
6         P(write_lock); // 第一个读者锁住写操作
7     }
8     V(reader_count_lock);
9     V(read_try); // (2) 释放
10
11     read_data();
```

```
12
13     P(reader_count_lock);
14     reader_count--;
15     if (reader_count == 0) {
16         V(write_lock); // 最后一个读者释放写锁
17     }
18     V(reader_count_lock);
19 }
```

写者逻辑:

```
1 void writer() {
2     P(writer_count_lock);
3     writer_count++;
4     if (writer_count == 1) {
5         P(read_try); // 第一个写者关门
6     }
7     V(writer_count_lock);
8
9     P(write_lock);
10    write_data();
11    V(write_lock);
12
13    P(writer_count_lock);
14    writer_count--;
15    if (writer_count == 0) {
16        V(read_try); // 最后一个写者开门
17    }
18    V(writer_count_lock);
19 }
```

关键点:

- read_try 是关键: 当 writer_count > 0 时, read_try 被第一个写者占用, 新读者无法进入。

- 写者等待所有已进入的读者退出后才能写（通过 P(write_lock)）。

公平策略 目标：读者与写者都不会饥饿，按请求先后顺序获得访问权。

表 4: 公平策略中的信号量与变量作用

信号量/变量名称	作用说明
queue	全局队列锁（初值为 1），确保请求 FIFO 排队，防止插队
wrt	读写互斥锁（初值为 1），实现读-写互斥与写-写互斥
mutex	保护 read_count 的互斥锁（初值为 1）
read_count	当前读者数量；当为 0 时释放 wrt

读者逻辑：

```
1 void reader() {
2     while (1) {
3         P(queue);
4         P(mutex);
5         read_count++;
6         if (read_count == 1) {
7             P(wrt);
8         }
9         V(mutex);
10        V(queue);
11
12        read_data();
13
14        P(mutex);
15        read_count--;
16        if (read_count == 0) {
17            V(wrt);
```

```
18     }  
19     V(mutex);  
20 }  
21 }
```

写者逻辑:

```
1 void writer() {  
2     while (1) {  
3         P(queue);  
4         P(wrt);  
5         V(queue);  
6  
7         write_data();  
8  
9         V(wrt);  
10    }  
11 }
```

关键点:

- queue 是公平策略核心: 所有请求先排队, 严格 FIFO。
- 读者拿到 queue 后尽快释放, 让后续请求继续排队。
- wrt 保证核心互斥: 读-写互斥、写-写互斥。

5.3.3 哲学家就餐问题

问题简述: 5 个哲学家围坐餐桌, 每两个相邻哲学家之间放 1 支筷子, 共 5 支。哲学家行为只有“思考”和“就餐”。就餐必须同时拿到左右两支筷子, 吃完放回。矛盾: 获取相邻资源可能导致循环等待进而死锁。

解决要点: 防止死锁, 确保至少一名哲学家可获得两支筷子。

方案: 最多 4 人同时就餐

信号量定义:

- semaphore room = 4;

- semaphore chopstick[5] = {1, 1, 1, 1, 1};

哲学家进程逻辑:

```
1 void philosopher(int i) {  
2     while(1) {  
3         think();  
4         P(room);  
5         P(chopstick[i]);  
6         P(chopstick[(i+1) % 5]);  
7         eat();  
8         V(chopstick[i]);  
9         V(chopstick[(i+1) % 5]);  
10        V(room);  
11    }  
12 }
```

工作原理:

- 房间容量限制: 最多允许 4 人同时进入 (通过 room 控制)。
- 5 支筷子最多 4 人持有, 至少 1 支筷子空闲, 因此至少 1 人能同时拿到两支筷子。
- 吃完释放资源, 其他哲学家可继续。

关键点:

- P(room) 必须在 P(chopstick) 之前, 先入场后拿筷子。
- 通过限制并发数量打破循环等待条件, 从根本上避免死锁。

5.3.4 睡眠理发师问题

问题简述: 理发店有 1 个理发师和若干等待椅。无顾客则理发师睡眠; 有顾客到来则叫醒理发师。若等待椅满, 新顾客离开。需确保理发师与顾客同步与互斥。

理发店模型关键点:

- 理发师无顾客时睡眠, 有顾客时被唤醒。

- 顾客到达时若有空椅子则等待，否则离开。
- 等待人数修改需互斥保护。

核心变量：

- `int waiting = 0;` 等待区顾客数（需互斥保护）。
- `#define N 5;` 等待椅数量（示例中 $N = 5$ ）。

表 5: 睡眠理发师问题的信号量设计

信号量名称	初值	作用
customers	0	顾客到达信号量：唤醒睡眠理发师
barbers	0	理发师准备好信号量：通知顾客可开始理发
mutex	1	互斥信号量：保护 <code>waiting</code> 修改

理发师进程：

```
1 void barber() {
2     while(1) {
3         P(customers);
4         P(mutex);
5         waiting--;
6         V(barbers);
7         V(mutex);
8         cut_hair();
9     }
10 }
```

顾客进程：

```
1 void customer() {
2     P(mutex);
3     if (waiting < N) {
4         waiting++;
5         V(customers);
```

```
6      V(mutex);
7      P(barbers);
8      get_haircut();
9  } else {
10     V(mutex);
11     leave();
12 }
13 }
```

关键点:

- `customers` 负责“顾客唤醒理发师”的同步。
- `barbers` 负责“理发师通知顾客可以理发”的同步。
- `mutex` 保护共享变量 `waiting`。

5.4 进程通信 (IPC) 机制

管道 (Pipe) 管道本质上是内核在内存中维护的缓冲区，允许一个进程写入，另一个进程读取。特性：自动同步（缓冲区满/空时阻塞）。例如命令行管道：`ls | grep "txt"`，`ls` 的输出通过管道传递给 `grep` 过滤，得到后缀为 `.txt` 的文件列表。

信号 (Signal) 信号是一种异步通知机制，用于通知进程某特定事件已发生；类似硬件中断，也称“软件中断”。

- 核心特点:

开销极小： 信号信息量少，通常仅一个整数（类型）。

异步性： 信号产生随机，进程通过注册处理函数被动响应。

不可靠性： 标准信号不排队，处理不过来可能丢失。

- 典型场景:

用户交互： 终端按下 `Ctrl+C`，操作系统向前台进程发送 `SIGINT`。

进程控制： 使用 `kill` 或系统调用发送 `SIGTERM/SIGKILL` 管理进程。

消息队列 (Message Queue) 消息队列允许进程间传递结构化数据，本质是内核中的消息链表。

- 核心特点：

结构化通信： 消息有边界与类型标识。

选择性接收： 接收方可按 Type ID 选择接收特定消息。

异步性： 消息可暂存在队列中。

- 典型场景：多级任务管理，如打印任务：普通任务为类型 1，紧急任务为类型 2，驱动可优先读取类型 2。

共享内存 (Shared Memory) 共享内存允许两个或多个进程直接访问同一物理内存区域，是 IPC 中速度最快的一种。

- 核心原理：

零拷贝 (Zero-Copy)： 不需要在用户空间与内核空间来回复制。

地址映射： 同一物理内存映射到不同进程虚拟地址空间。

- 关键挑战：同步问题
- 适用场景：高性能且大数据量传输（如实时视频、高频交易）。

6 死锁

6.1 基本概念

6.1.1 定义与产生原因

死锁：一组进程/线程中每个都在等待其他释放资源，从而形成无限等待。

死锁产生根本原因：

- 竞争不可抢占资源：资源数量有限；进程占有资源 A 并申请资源 B ，而 B 被他人占有且不可强制剥夺时进程等待；若形成环路则僵局。
- 进程推进顺序不当：申请与释放资源的时机与顺序不当（形成循环等待链）导致死锁。

6.1.2 产生死锁的必要条件

- 互斥条件 (**Mutual Exclusion**)：某资源同一时间仅为一个进程占有。
- 持有并等待 (**Hold and Wait**)：占有资源同时申请新资源。
- 不可抢占 (**No Preemption**)：资源只能由占有者主动释放。
- 循环等待 (**Circular Wait**)：存在进程资源循环等待链。

6.1.3 资源分配图

资源分配图 (Resource Allocation Graph, RAG) 是描述资源分配状态的有向图，由节点集合 V 与边集合 E 组成，是检测死锁的重要工具。

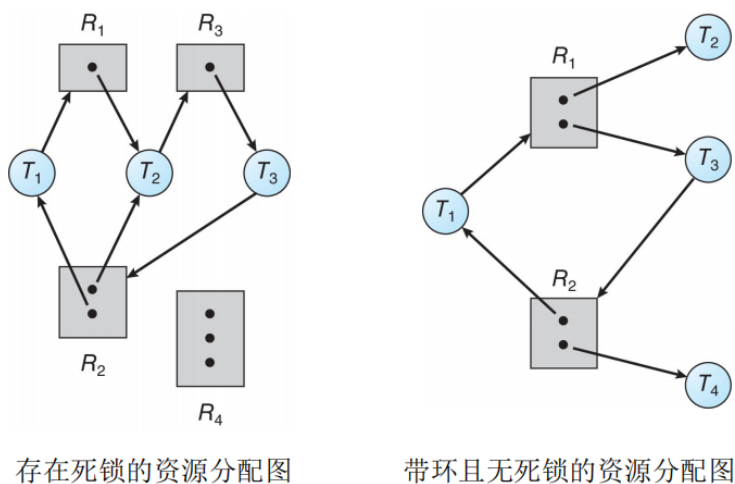


图 8: 资源分配图示例

- 节点集合 V :

进程节点集合 P : $\{P_1, P_2, \dots, P_n\}$ (圆形)。

资源节点集合 R : $\{R_1, R_2, \dots, R_m\}$ (方框); 方框内圆点数量表示资源实例数。

- 边集合 E :

请求边 (**Request Edge**): $P_i \rightarrow R_j$ 。

分配边 (**Assignment Edge**): $R_j \rightarrow P_i$ 。

死锁判定法则:

1. 无环: 资源分配图无环则不存在死锁。
2. 有环: 需结合实例数判断。
 - 每类资源仅 1 个实例: 有环为死锁充分必要条件。
 - 每类资源有多个实例: 有环为死锁必要条件, 可能存在死锁。

6.2 处理策略

6.2.1 死锁预防：破坏必要条件

死锁预防通过设计限制破坏四个必要条件之一。

- 破坏互斥条件：使资源可共享（如 SPOOLing 将独占打印机改造为共享）；但适用范围有限。
- 破坏持有并等待：静态资源分配，一次性申请全部资源；缺点是资源利用率低且可能饥饿。
- 破坏不可抢占：申请新资源被阻塞时释放已占资源，后续再重新申请；实现复杂且不适用于某些设备。
- 破坏循环等待：有序资源分配，为资源编号并规定按编号递增申请；广泛采用但限制灵活性。

6.2.2 死锁避免：银行家算法 (安全序列)

银行家算法在每次分配前检查分配后系统是否处于安全状态；安全则分配，否则等待。

关键概念：

- 安全状态：存在进程序列 $\langle P_1, \dots, P_n \rangle$ 使得各进程依次获得所需资源并完成。
- 不安全状态：无法找到序列，可能（但不一定）导致死锁。
- 安全序列：对序列中每个进程 P_i ，其剩余需求可由“当前可用资源 + 之前释放资源”满足。

数据结构定义：设 n 个进程 $\{P_0, \dots, P_{n-1}\}$ ， m 类资源 $\{R_0, \dots, R_{m-1}\}$ ：

- **Available**：长度 m 的向量，表示各类资源剩余数量。
- **Max**： $n \times m$ 矩阵，表示每进程最大需求。
- **Allocation**： $n \times m$ 矩阵，表示已分配资源。
- **Need**： $n \times m$ 矩阵， $\text{Need}[i][j] = \text{Max}[i][j] - \text{Allocation}[i][j]$ 。

银行家算法执行步骤：

1. 进程 P_i 发起请求，向量为 Request_i 。
2. 合法性检查：
 - 若 $\text{Request}_i[j] > \text{Need}[i][j]$ （任意 j ），请求非法。
 - 若 $\text{Request}_i[j] > \text{Available}[j]$ （任意 j ），资源不足，进程等待。
3. 预分配（临时修改数据结构）：

$$\text{Available}[j] \leftarrow \text{Available}[j] - \text{Request}_i[j]$$

$$\text{Allocation}[i][j] \leftarrow \text{Allocation}[i][j] + \text{Request}_i[j]$$

$$\text{Need}[i][j] \leftarrow \text{Need}[i][j] - \text{Request}_i[j]$$

4. 安全性检查：寻找安全序列。
5. 若安全则正式分配，否则回滚并拒绝。

示例：

进程	Max			Allocation			Need		
	A	B	C	A	B	C	A	B	C
P_0	7	5	3	0	1	0	7	4	3
P_1	3	2	2	2	0	0	1	2	2
P_2	9	0	2	3	0	2	6	0	0
P_3	2	2	2	2	1	1	0	1	1
P_4	4	3	3	0	0	2	4	3	1

当前可用资源： $\text{Available} = [3, 3, 2]$ 。

6.2.3 死锁检测：化简资源分配图

具体参考上文资源分配图部分内容。

6.2.4 死锁解除

1. 进程终止 (Process Termination)

- 终止所有死锁进程：最简单但代价大。
- 逐个终止进程：每次终止一个并重新检测，直到死锁消失。
- 选择策略：考虑优先级、已运行时间、已占资源、仍需资源等。

2. 资源抢占 (Resource Preemption)

- 选择牺牲者：选择代价最小的进程剥夺资源。
- 回滚：将其回滚到某安全状态（最简单是重启）。
- 防止饥饿：避免同一进程总被选为牺牲者。

7 内存管理

7.1 基础概念

- 逻辑地址：程序视角下的地址，也称相对地址；程序通常认为从地址 0 开始。
- 物理地址：硬件视角下的绝对地址。例如基址为 1000，逻辑地址 500 对应物理地址 1500。
- 重定位：将逻辑地址转换为物理地址。

静态重定位 (Static Relocation): 编译器或加载器直接修改指令地址为绝对地址；程序装入后不能移动，不支持多道。

动态重定位 (Dynamic Relocation): 运行时转换，依赖重定位寄存器（基址寄存器）；公式：物理地址 = 逻辑地址 + 基址寄存器值。

- 内存保护 (Memory Protection): 使用界限寄存器 (Limit Register) 防止越界；检查逻辑地址是否小于界限。

- 覆盖与交换技术

1. 覆盖 (Overlay)

- 核心场景：单个程序，程序大于物理内存。
- 基本原理：常驻区放核心代码；覆盖区放互斥模块，共用内存。
- 示意图：



- 例：程序总大小 190KB，内存 110KB。常驻区 A(20KB)；覆盖区 0 放 B(50KB) 与 C(30KB) 互斥；覆盖区 1 放 D/E/F 中最大 E(40KB)。总需求 $20 + 50 + 40 = 110\text{KB}$ 。
- 优点：大程序可在小内存运行。
- 缺点：程序员需手动规划模块关系。

2. 交换 (Swapping)

- 核心场景：多道程序环境。

- 基本原理：将暂时不用的进程整体换出到外存，对换出 (Swap Out) 与换入 (Swap In)。
- 特点：透明性强，支持多道。
- 缺点：硬盘 I/O 开销大，性能较低。

7.2 内存分配方案

7.2.1 连续分配

1. 固定分区分配 (Fixed Partitioning)
- 原理：内存划分为固定大小分区，每分区容纳一个进程。
 - 优点：实现简单。
 - 缺点：利用率低、碎片多；分区数量固定、大小死板。
2. 动态分区分配 (Dynamic Partitioning) 按程序需求动态分配，常见管理结构：空闲分区表与空闲分区链表。



分区号	大小	起始地址
1	8KB	24KB
2	12KB	128KB
3	8KB	248KB
4	...	...
5	...	...

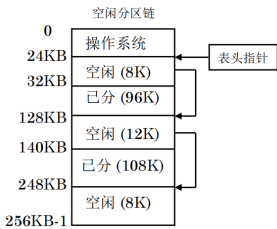


图 9: 内存布局图

图 10: 空闲分区表

图 11: 空闲分区链

动态分区存在外部碎片：空闲分区总量足够但物理位置不连续。

- (a) 紧凑化 (Compaction)：移动所有进程，开销大。
- (b) 分配算法优化：
 - 首次适应 (First Fit)
 - 循环首次适应 (Next Fit)

- 最佳适应 (Best Fit)
 - 最差适应 (Worst Fit)
3. 伙伴系统 (Buddy System) 空闲块大小为 2^k ，分配时找到最小满足需求块，不足则拆分；回收时尽可能与伙伴合并。
- 优点：回收效率高、灵活。
 - 缺点：内部碎片；拆分/合并开销。

7.2.2 离散分配

1. 分页管理 (Paging)

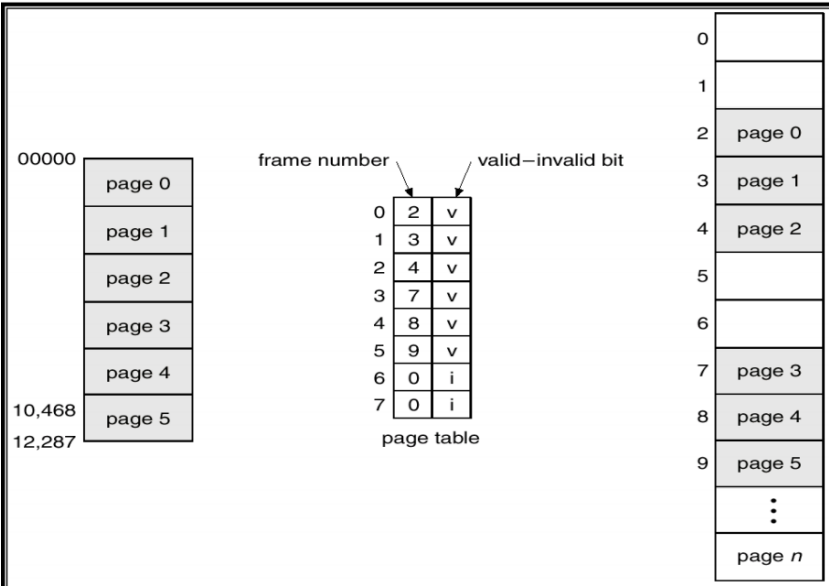


图 12: 分页管理示意图

页表（如图 12）：逻辑页面 → 物理块；页面 → 页框。地址计算：

物理地址 = 页框号 × 页大小 + 页内偏移

快表 (TLB)：缓存最近使用的页表项。

表 6: 页表与快表比较

特性	页表 (Page Table)	快表 (TLB)
位置	主内存 (RAM)	CPU 内部 (MMU)
介质	DRAM	SRAM (联想存储器)
容量	大, 存放全部页表项	小, 通常只有 64-1024 项
访问速度	慢	极快
查找方式	索引查找	并行比较查找

有效时间计算：设一次内存访问时间为 m ，快表查找时间为 n ，命中率为 p ，则

$$EAT = (n + m) \times p + (n + 2m) \times (1 - p) = n + m + (1 - p) \times m.$$

2. 分段管理 (Segmentation)

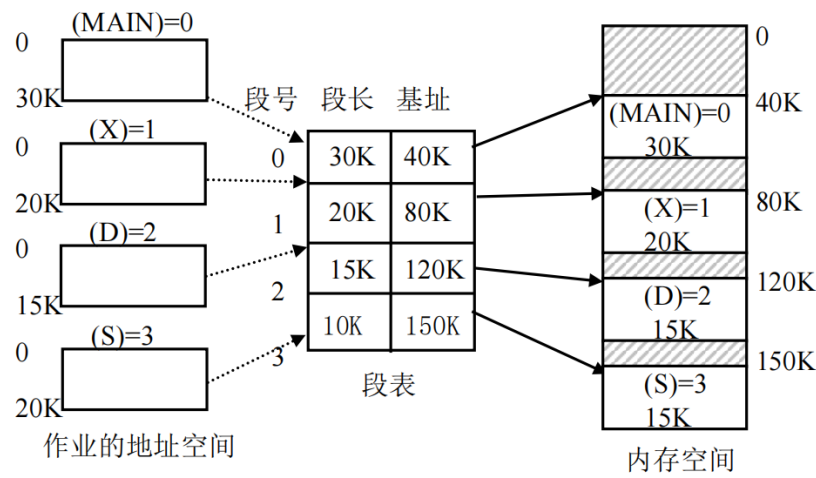


图 13: 分段管理示意图

3. 段页式管理 (Segmented Paging)

7.3 虚拟内存技术

虚拟内存将用户逻辑内存与物理内存分开，使程序可使用比实际物理内存更大的地址空间。

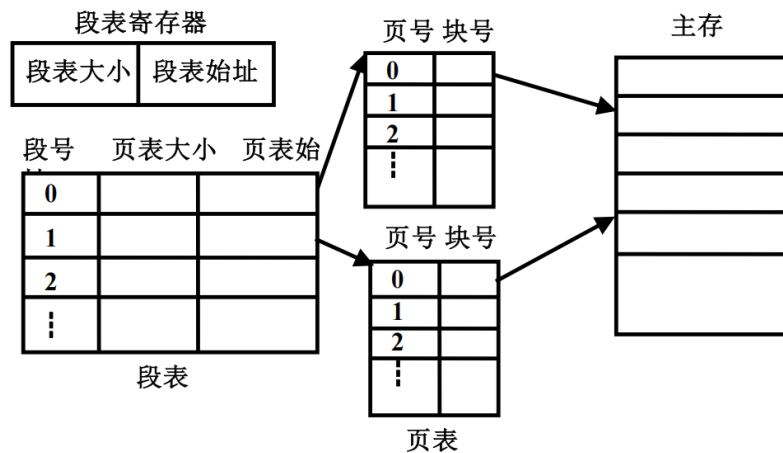


图 14: 段页式管理示意图

局部性原理:

- 时间局部性: 最近访问的数据/指令可能不久再访问。
- 空间局部性: 相邻地址的数据/指令可能不久被访问。

虚拟存储器实现方法:

- 请求分页 (Demand Paging)
 1. 扩充页表结构: 有效位、修改位、访问位、外存位置指针等。
 2. 页错误: 访问页不在内存时触发。
 3. 缺页中断: 处理页错误的过程。
 4. 有效访问时间 (EAT): 设内存读写周期为 t , 缺页中断服务时间为 t_1 , 快表命中率为 α , 缺页率为 f , 快表访问时间为 ε :

$$EAT = \alpha(t + \varepsilon) + (1 - \alpha)[(1 - f) \cdot 2(t + \varepsilon) + f \cdot (t_1 + 2(t + \varepsilon))].$$

5. 页面置换算法: FIFO、OPT、LRU、CLOCK 等。
 6. 页面分配算法: 平均分配、按比例分配、按优先级分配。
 7. 抖动 (Thrashing): 多数时间用于换入/换出。
 8. 工作集模型: 工作集窗口 Δ , 工作集大小 WSS 等。
- 请求分段 (Demand Segmentation)

7.4 优化技术

写时复制 (COW): 没人修改则共享同一页; 一旦写入则复制出私有页。

1. 父子进程共享同一页并标记为只读。
2. 写入触发异常。
3. 操作系统捕获异常, 为写入方复制新页。
4. 仅复制被修改页, 其余页继续共享。

8 文件系统

8.1 文件系统基础

- 扇区: 磁盘最小存储单位, 通常为 512 字节。

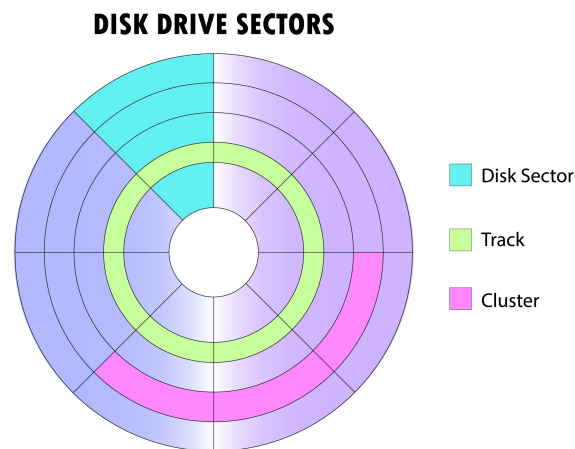


图 15: 扇区示意图

- 元数据: 描述文件属性的信息, 如标识、位置、属性与时间等。
 - Inode: 存储文件元数据与磁盘块地址 (固定大小)。
 - Dirent: 目录项, 包含文件名与对应 Inode 号。

- 视图转换：依靠 Inode（静态索引地图）与 bmap（动态查找算法）实现，将逻辑视图转换为物理视图。

8.2 文件组织方式

8.2.1 空闲空间管理

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	1	1	0	0	1	1	0	1	1	1	0	1	1	1	1	1
1	0	0	0	0	1	1	1	1	1	0	0	0	0	0	0	1
2	1	1	1	1	1	1	0	1	1	1	1	0	0	0	0	0
3																
4	...															
⋮																

图 16: 位图法示意图

1. 位图法 (Bitmap Method)

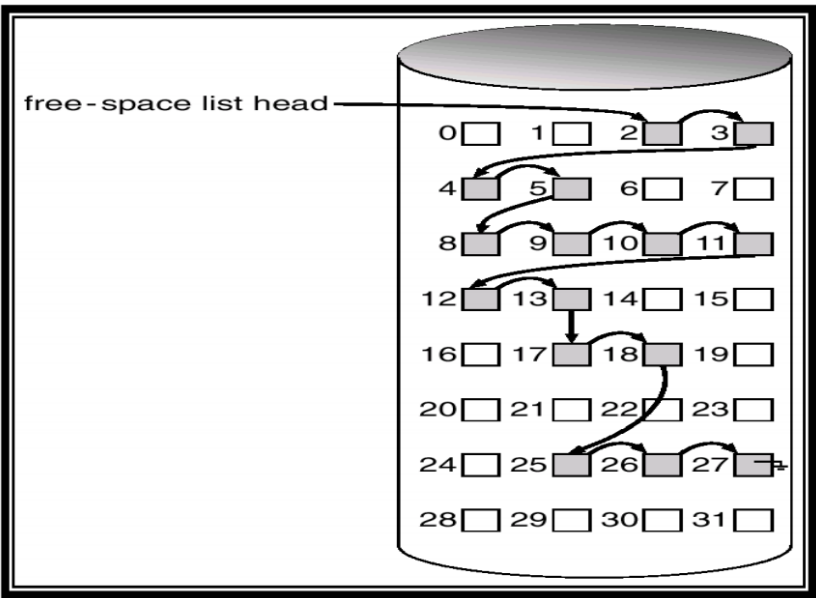


图 17: 空闲链表法示意图

2. 空闲链表法 (Linked List Method)

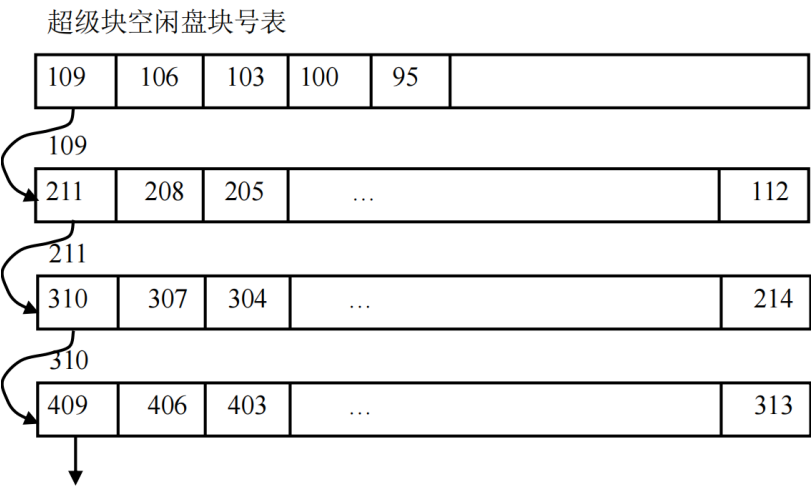


图 18: 成组链接法示意图

3. 成组链接法 (Grouped Link Method)

8.2.2 接口设计

文件描述符作为凭证，Open 建立连接，Close 断开连接。

1. 文件描述符 (File Descriptor, fd)

- 定义：非负整数（如 3, 4, 5, ...）。
- 作用：用户进程与内核文件结构之间的抽象句柄。
- 本质：当前进程“打开文件表”数组的下标。

2. Open 方法

- 原型：int Open(char *path, int mode);。
- 内部流程：寻址、权限检查、创建 file 状态、分配 fd。

3. Close 方法

- 原型：int Close(int fd);。
- 内部流程：回收 fd、减少引用计数、断开与 Inode 的关联。

8.2.3 连续分配

连续分配要求文件占用连续盘块。

- 优点：顺序访问容易且速度快；目录项简单。
- 缺点：易碎片；难找连续空间；文件难增长。

8.2.4 链接分配

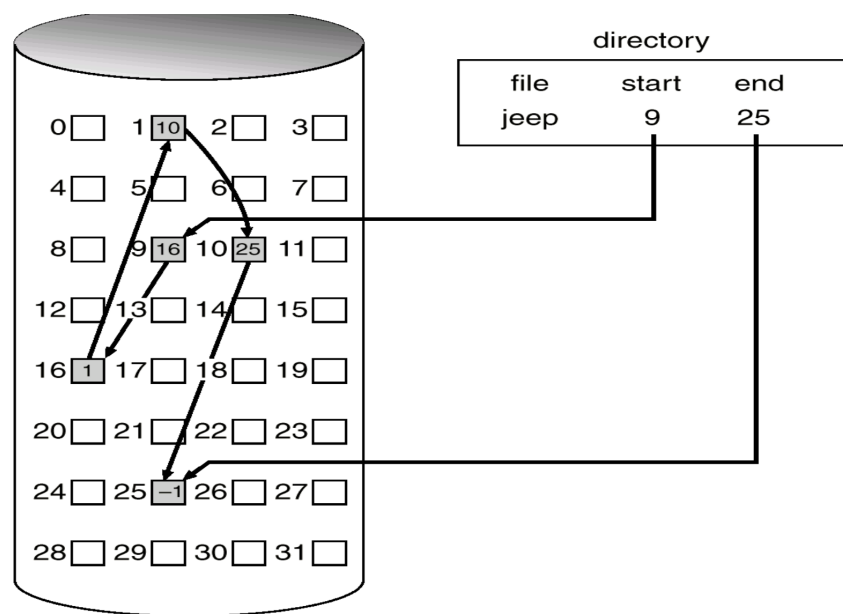


图 19: 链接分配示意图

文件分配表 (FAT)

FAT 表大小 = 磁盘块总数 × 每个表项大小.

每个表项大小 (字节) = $\lceil \log_2(\text{磁盘块总数})/8 \rceil$.

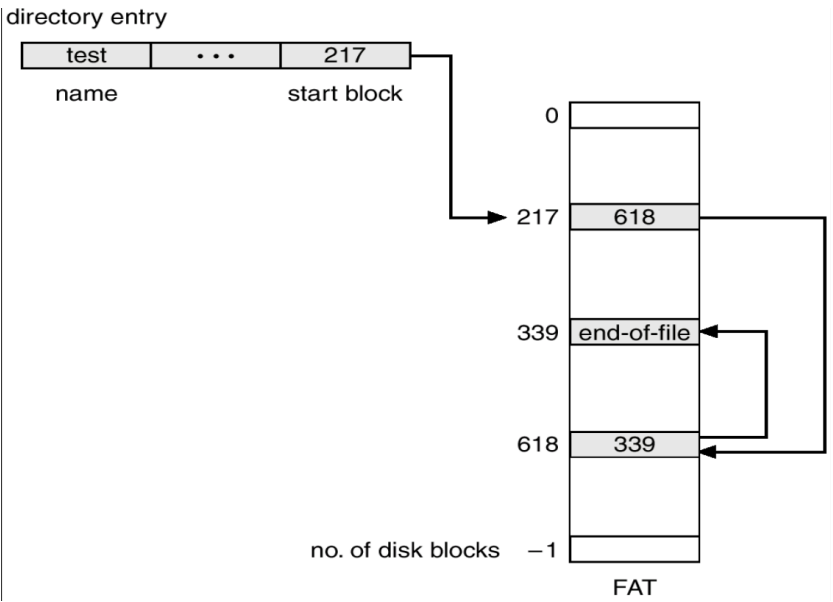


图 20: FAT 文件分配表示意图

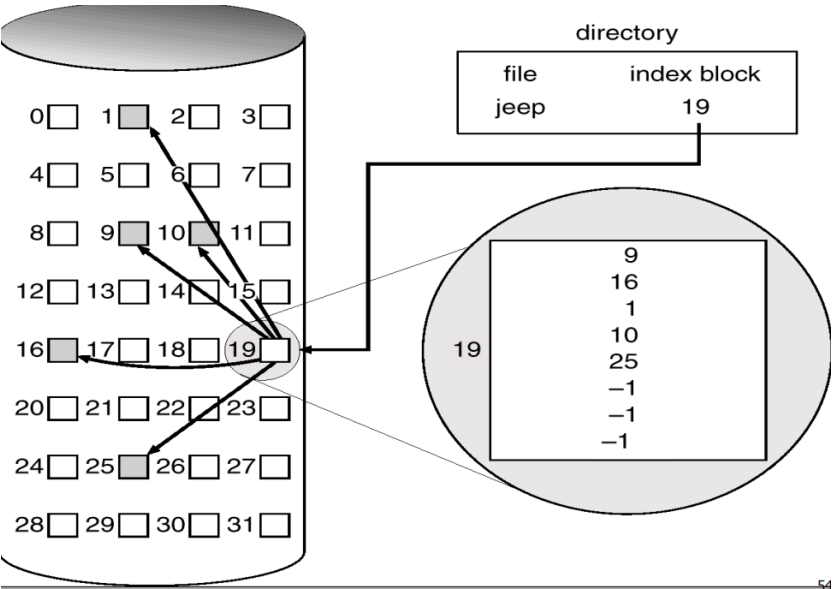


图 21: 索引分配示意图

8.2.5 索引分配

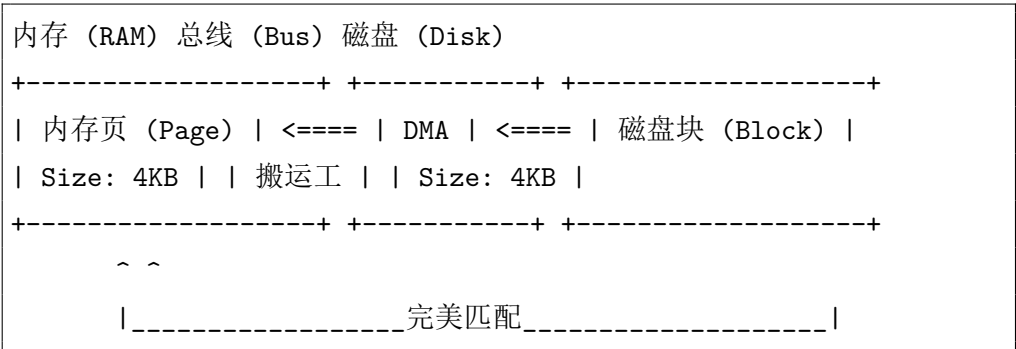
8.3 文件目录与路径

8.3.1 文件链接

- 1. 硬链接 (**Hard Link**): 多个目录项指向同一 Inode; 引用计数为 0 时删除。
- 2. 软链接/符号链接 (**Symbolic Link**): 独立文件, 内容为目标路径; 可跨文件系统。

8.4 文件系统性能

- 1. 缓存技术 (**Caching**): 减少磁盘访问。
 - 缓冲区缓存 (Buffer Cache): 缓存磁盘块。
 - 页面缓存 (Page Cache): 缓存文件数据页。



- 2. 预读与写回 (**Read-Ahead and Write-Back**): 检测顺序访问并提前读取后续块。
- 3. 延迟写 (**Delayed Write**): 写操作先缓存在内存, 条件满足时批量写回。

8.5 虚拟文件系统 (VFS)

VFS 是应用程序与具体文件系统实现之间的软件抽象层, 提供统一文件系统接口 (POSIX)。

- 超级块 (**super_block**): 表示挂载文件系统。
- 索引节点 (**inode**): 文件元数据。
- 目录项 (**dentry**): 目录中文件/子目录项。
- 文件对象 (**file**): 打开文件的状态与读写指针。

9 设备管理

9.1 基本概念

I/O 设备及分类:

1. 按使用特性:

- 人机交互设备: 键盘、鼠标、显示器。
- 存储设备: 硬盘、U 盘、光驱。
- 网络通信设备: 网卡、调制解调器。

2. 按传输速率:

- 低速: 键盘 (10B/s)、鼠标 (100B/s)。
- 中速: 打印机 (100KB/s)。
- 高速: 磁盘 (100MB/s)、网卡 (1GB/s)。

3. 按信息交换单位:

- 块设备: 硬盘、U 盘 (可随机访问)。
- 字符设备: 键盘、串口 (流式访问)。
- 网络设备: 网卡 (数据包)。

I/O 端口: 设备控制器负责在 CPU/内存与设备之间做信号转换; I/O 端口是控制器与 CPU/内存交换数据的接口。

端口类型	作用
数据端口 (Data)	存放输入/输出数据（缓冲）
状态端口 (Status)	指示设备忙/闲、错误（握手）
控制端口 (Ctrl)	接收启动、复位等命令

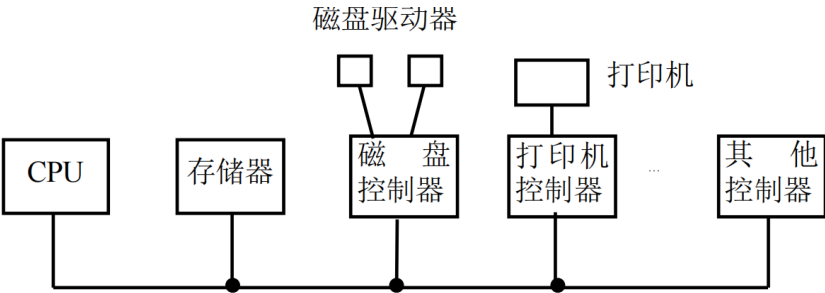


图 22: 总线示意图

I/O 控制方式:

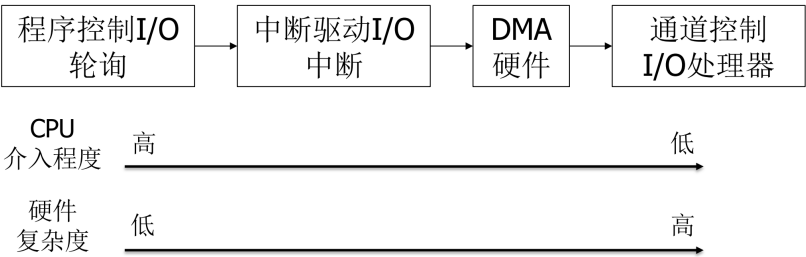


图 23: I/O 控制方式示意图

- 1. 程序控制方式 (Programmed I/O): CPU 轮询设备状态。
 - 优点: 实现简单。
 - 缺点: CPU 忙等, 效率低。
- 2. 中断驱动方式 (Interrupt-Driven I/O): 设备完成后中断通知 CPU。
 - 优点: 提高 CPU 利用率。

- 缺点：每次中断都需 CPU 介入，开销较大。
3. DMA 方式 (Direct Memory Access): DMA 控制器搬运数据，完成后中断通知 CPU。
 - 优点：大幅减少 CPU 介入，适合大批量传输。
 4. 通道控制方式 (Channel I/O): 通道控制器可执行一系列 I/O 指令，统筹多台设备。

9.2 硬盘

9.2.1 机械硬盘

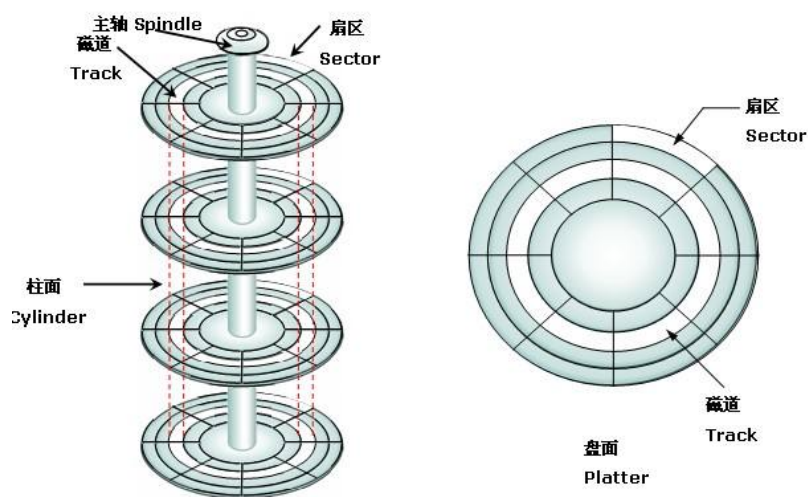


图 24: 机械硬盘结构示意图

机械硬盘 (HDD) 通过盘片旋转与磁头寻址存储数据。

总访问时间 = 寻道时间 + 旋转延迟 + 传输时间。

9.2.2 固态硬盘

SSD 无机械部件。

- NAND 闪存芯片：存储介质

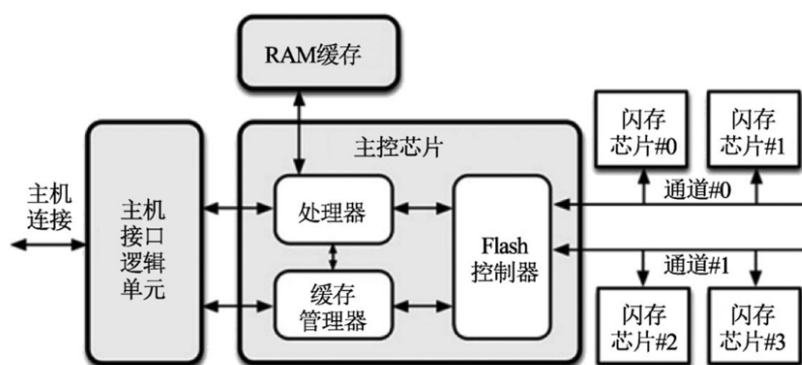


图 25: 固态硬盘结构示意图

- 控制器：管理读写与算法
- DRAM 缓存：提高速度

FTL (Flash Translation Layer): 解决 OS 逻辑块地址 (LBA) 与 NAND 物理特性 (PPA、写前擦除) 的矛盾。

1. 地址映射：LBA $\rightarrow$ PPA。
2. 垃圾回收：回收无效页。
3. 磨损均衡：均匀写入次数。
4. 写放大效应：实际写入量大于请求写入量。

9.3 I/O 系统层次架构

- 硬件设备层：各种物理 I/O 设备（磁盘、键盘、显示器等）。
- 中断处理程序层：响应设备中断请求，通知操作系统 I/O 事件发生。
- 设备驱动程序层：为具体设备提供操作接口，封装硬件细节。
- 设备独立内核软件层：提供统一 I/O 接口，屏蔽不同驱动差异。
- 系统调用接口层：为用户进程提供标准 I/O 接口（read、write）。
- 输入输出程序库层：为应用程序提供更高级 I/O 封装（文件库等）。

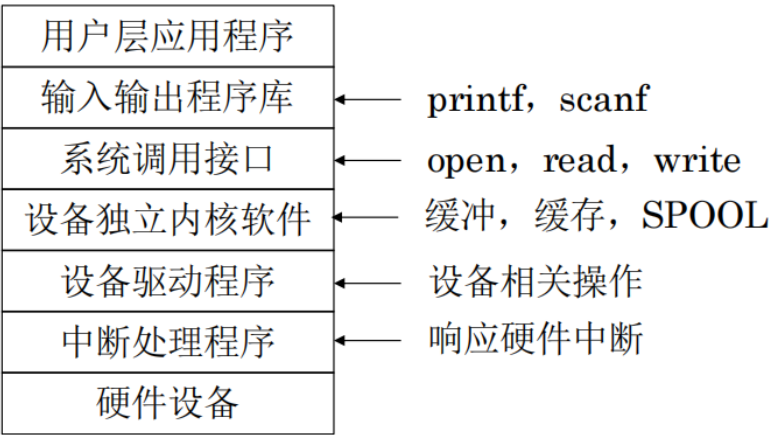


图 26: I/O 系统层次架构示意图

9.4 中断处理程序

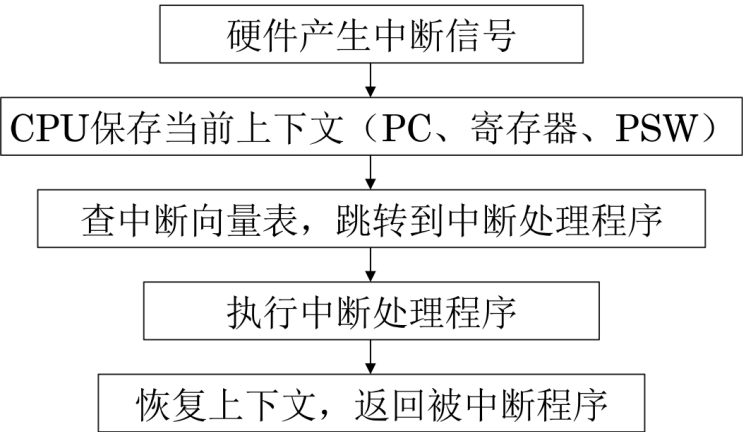


图 27: 中断处理程序示意图

9.4.1 上下文保护与恢复

保存内容:

- 程序计数器 (PC / IP): 记录被打断时执行位置。

- 程序状态字 (PSW / EFLAGS): 记录运算状态与权限等级等。
- 通用寄存器: 记录中间变量。
- 栈指针 (SP): 记录调用栈位置。

保存位置: 内核栈。保存方式:

1. 硬件自动保存: 响应中断时自动将 PC 与 PSW 压栈。
2. 软件手动保存: 处理中断时保存通用寄存器与 SP。

9.4.2 上半部与下半部机制

为何分离: 中断处理应尽可能快; 复杂工作可延迟; 避免在中断上下文做复杂操作。

表 7: 上半部与下半部机制对比		
特性	上半部 (Top Half)	下半部 (Bottom Half)
别名	硬中断 (Hardirq)	软中断 (Softirq) / 延迟处理
执行时机	中断发生后立刻执行	稍后执行 (系统空闲或中断退出前)
中断状态	关闭中断	开启中断
主要任务	签收信号、拷贝少量数据、 登记下半部	复杂处理、协议栈解析、磁 盘 I/O
耗时要求	极快 (微秒级)	可以较长
能否睡眠	禁止	Workqueue 方式可睡眠

9.5 设备独立内核软件技术

缓冲区 (Buffer) 是临时存储区域, 用于弥补生产者与消费者速度差异。

9.5.1 缓冲技术

单缓冲

- 输入阶段: 设备写入缓冲区。

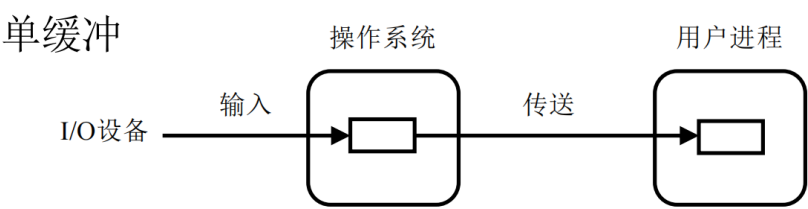


图 28: 单缓冲示意图

- 传送阶段：操作系统将数据从缓冲区搬到用户区。
- 处理阶段：CPU 处理用户区数据。

设设备传输时间为 M ，CPU 处理时间为 C ，单缓冲处理一块数据时间为 $M + C$ 。

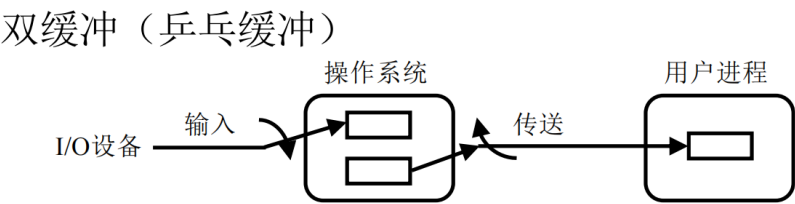


图 29: 双缓冲示意图

双缓冲 双缓冲允许设备与 CPU 并行工作，处理一块数据时间近似为 $\max(C, M)$ 。

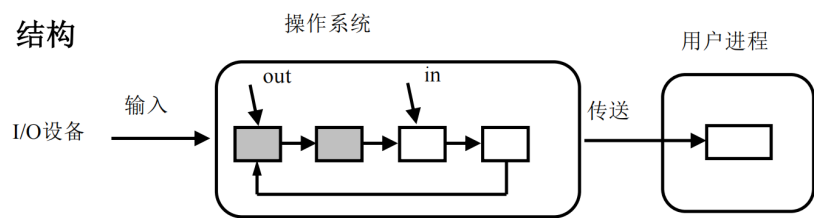


图 30: 环形缓冲示意图

环形缓冲 空： $out == in$ ；满： $(in + 1) \% N == out$ 。

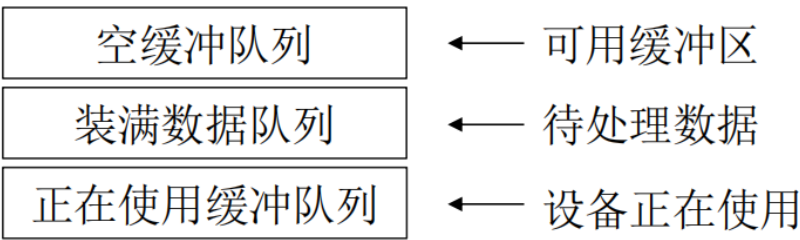


图 31: 缓冲池示意图

缓冲池

- 1. 设备从空闲队列获取缓冲区。
- 2. 填充数据后放入输入队列。
- 3. CPU 从输入队列取出处理。
- 4. 处理完放回空闲队列。

表 8: 缓冲与缓存对比

维度	缓冲 (Buffer)	缓存 (Cache)
核心目的	平滑速度差异	加速访问、减少延迟
数据使用	通常一次	可能多次
方向	多为单向传输链路	双向数据复用
典型策略	FIFO	LRU

9.5.2 异步 I/O (AIO)

表 9: 同步 I/O 与异步 I/O 对比

维度	同步阻塞 I/O	异步 I/O
调用后行为	线程阻塞等待完成	函数立即返回
通知机制	完成后唤醒线程	信号或回调通知
编程复杂度	简单直观	较复杂（状态管理）

9.5.3 SPOOLing 技术 (假脱机)

SPOOLing 用于解决独占设备共享与 CPU/外设速度差异。

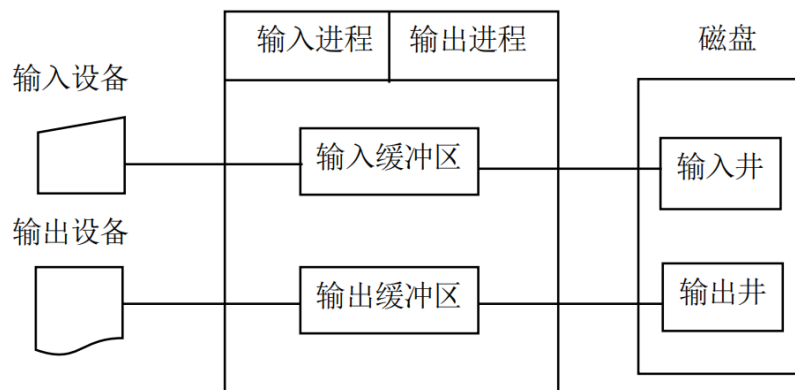


图 32: SPOOLing 系统结构示意图

优点:

- 提高设备利用率: 将独占设备虚拟为共享设备。
- 提高 CPU 效率: 进程无需等待慢速设备。

缺点:

- 需要额外磁盘空间作缓冲。
- 系统复杂度增加。

9.5.4 设备分配与回收

设备分类:

1. 独占设备: 一次只能被一个进程使用 (如打印机)。
2. 共享设备: 可同时被多个进程使用 (如磁盘)。
3. 虚拟设备: 通过 SPOOLing 将独占设备虚拟化为共享设备。